

Module 11 — Observability

“You can’t fix what you can’t see.” Observability = **metrics + logs + traces** (the three pillars). Interviewers want the Spring stack (Actuator + Micrometer + Prometheus + Grafana) and how you trace a request across microservices.

11.1 The Three Pillars

Pillar	Answers	Tooling
Metrics	how much / how fast (aggregates)	Micrometer → Prometheus → Grafana
Logs	what happened (discrete events)	SLF4J/Logback → ELK/Loki
Traces	where time went across services	Micrometer Tracing / OpenTelemetry → Jaeger/Tempo/Zipkin

Monitoring = watching known metrics/alerts; **observability** = ability to ask new questions about unknown failure modes from your telemetry.

11.2 Actuator & Micrometer

Actuator

Exposes operational endpoints (Module 3.9): `/actuator/health` (probes), `/metrics`, `/prometheus`, `/loggers`, `/threaddump`, `/heapdump`, `/httpexchanges`.

Micrometer

A **vendor-neutral metrics facade** (“SLF4J for metrics”) built into Spring Boot. You instrument once; export to Prometheus, Datadog, CloudWatch, etc.

Meter types

- **Counter** — monotonically increasing (requests, errors).
- **Gauge** — current value (queue size, cache entries, pool active).
- **Timer** — count + total time + percentiles (latency).
- **DistributionSummary** — distributions (payload sizes).
- **Tags/labels** — dimensions (`uri`, `status`, `method`) — keep cardinality low!

Real Production Example

```
Timer.Sample s = Timer.start(registry);
try { doWork(); }
finally { s.stop(registry.timer("order.process", "type", type)); }

meterRegistry.counter("orders.created", "channel", channel).increment();
```

Boot auto-instruments HTTP (`http.server.requests`), JVM (memory/GC/threads), HikariCP, Kafka, etc.

Common Mistakes

High-cardinality tags (user id, request id in labels) blow up Prometheus memory/series. Use bounded label values; put unbounded ids in traces/logs.

Interview Q

- Micrometer purpose; counter vs gauge vs timer.
- Why is metric cardinality dangerous?
- What does `http.server.requests` give you (RED metrics)?

11.3 Prometheus & Grafana

- **Prometheus** — time-series DB that **scrapes** `/actuator/prometheus` periodically; stores metrics; **PromQL** for queries; **Alertmanager** for alerts.
- **Grafana** — dashboards/visualization over Prometheus (and logs/traces).
- **RED method** (request-rate, errors, duration) for services; **USE** (utilization, saturation, errors) for resources.

ASCII

```

app /actuator/prometheus <--scrape-- Prometheus --query--> Grafana dashboards
                        |
                        Alertmanager -> Slack/PagerDuty

```

Interview Q

Pull (Prometheus scrape) vs push model; what to alert on (SLOs: latency, error rate, saturation).

11.4 Centralized Logging

Core Concept

Aggregate logs from all instances/services into one searchable store (ELK: Elasticsearch+Logstash+Kibana, or Loki+Grafana). Use **structured JSON logs** + **correlation/trace id in MDC** so you can follow one request across services.

Best Practices

- JSON encoder (logback) with fields: timestamp, level, logger, traceId, spanId, service, message.
- Put `traceId` in MDC via a filter; propagate downstream (headers).
- Log levels appropriate; never log secrets/PII; sample noisy logs.

Interview Q

Why structured logging; how to correlate logs across microservices (trace id in MDC); log vs metric vs trace — when to use which.

11.5 Distributed Tracing & OpenTelemetry

Why Interviewers Ask This

In microservices, one user request fans out to many services; tracing shows the end-to-end path and where latency is spent.

Core Concept

- A **trace** = one request's journey; composed of **spans** (units of work). Each span has a `traceId` (shared), `spanId`, parent span id, timing, tags.
- **Context propagation**: the trace context travels via headers (W3C `traceparent`) across HTTP/Kafka so spans link up.

- **Micrometer Tracing** (successor to Spring Cloud Sleuth) + **OpenTelemetry** (vendor-neutral standard for traces/metrics/logs) export to Jaeger/Tempo/Zipkin.

ASCII — a trace

```

traceId=abc
┌ gateway [span 1] ─────────────────── 120ms
│ ┌ order-service [span 2] ─────────── 90ms
│ │ ┌ DB query [span 3] ─ 20ms
│ │ └ payment-service [span 4] ─ 55ms <-- bottleneck
│ └ ...
└ ...

```

Real Production Example

p99 latency spikes on checkout. The trace shows most time in **payment-service**'s external call → add timeout/circuit breaker there and cache pricing.

Interview Q / Follow-ups

- What are traces and spans; how is context propagated (traceparent)?
- Sleuth vs Micrometer Tracing vs OpenTelemetry.
- How would you find which service causes a latency spike? (*distributed trace*.)
- Sampling — why (cost) and head vs tail sampling.

Hands-on Exercise

Add Micrometer Tracing + OTel exporter; make two services call each other; confirm one trace with linked spans and a propagated **traceId** in logs.

Module 11 — One-Page Cheat Sheet

Topic	Key point
Pillars	metrics (aggregate), logs (events), traces (cross-service path)
Actuator	health/metrics/prometheus/loggers/threaddump
Micrometer	metrics facade; counter/gauge/timer; low cardinality!
Prometheus	scrapes /prometheus; PromQL; Alertmanager
Grafana	dashboards; RED (rate/errors/duration), USE
Logging	structured JSON + traceId in MDC; never log secrets
Tracing	trace=spans; W3C traceparent propagation; OTel standard
Probes	liveness vs readiness via health groups

Module 11 — Top Interview Questions

1. Three pillars of observability; when use each.
2. What is Micrometer; meter types; cardinality pitfall.
3. How does Prometheus collect metrics (pull/scrape)?
4. How do you correlate logs across microservices?
5. Traces vs spans; context propagation.
6. Sleuth vs Micrometer Tracing vs OpenTelemetry.
7. How to find a latency bottleneck across services.
8. What to alert on (SLOs); RED/USE.
9. Liveness vs readiness probes.

10. Monitoring vs observability.

Module 11 — Common Mistakes

- High-cardinality metric labels (ids) → Prometheus OOM.
- Unstructured logs; no trace id.
- Logging secrets/PII.
- Alerting on causes instead of symptoms (SLOs).
- No sampling → tracing cost explosion.

Module 11 — Mock Interview

1. *"Checkout p99 spiked; where do you look?"* → distributed trace to find the slow span/service; correlate with metrics; check that service's dependencies.
2. *"How do you follow one request across 6 services in logs?"* → propagate a traceId (W3C traceparent) into MDC; structured JSON logs; search by traceId.
3. *"Prometheus is OOMing."* → high-cardinality labels (user/request ids); remove them, keep dimensions bounded.
4. *"What alerts would you set for a service?"* → SLO-based: error rate, p99 latency, saturation (RED/USE), not raw CPU alone.
5. *"Liveness vs readiness probe?"* → liveness = restart if dead; readiness = remove from LB until ready/deps healthy.

Next → Module 12: Docker & Kubernetes.